

Artificial Intelligence

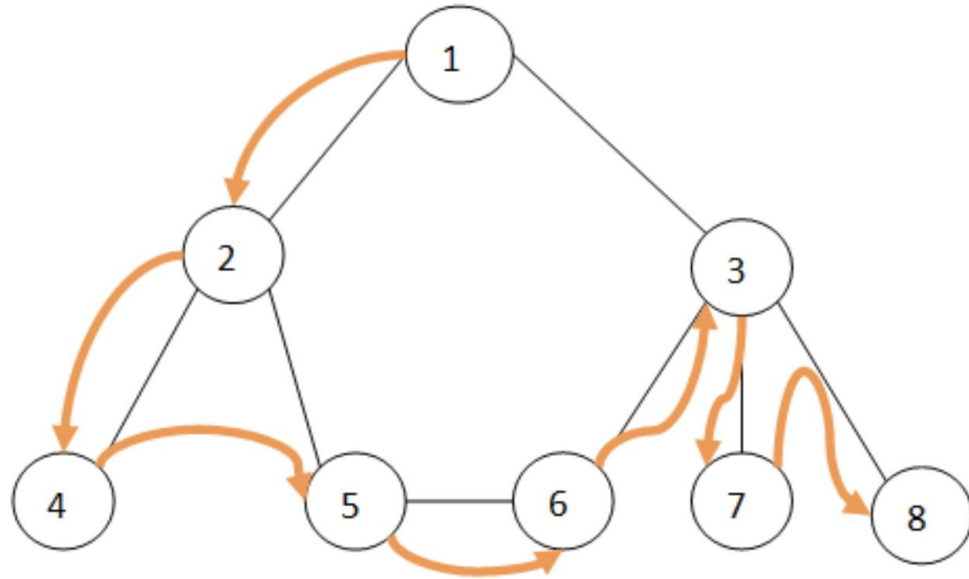
Section 6

By: Mohamed Fathy

Section Content

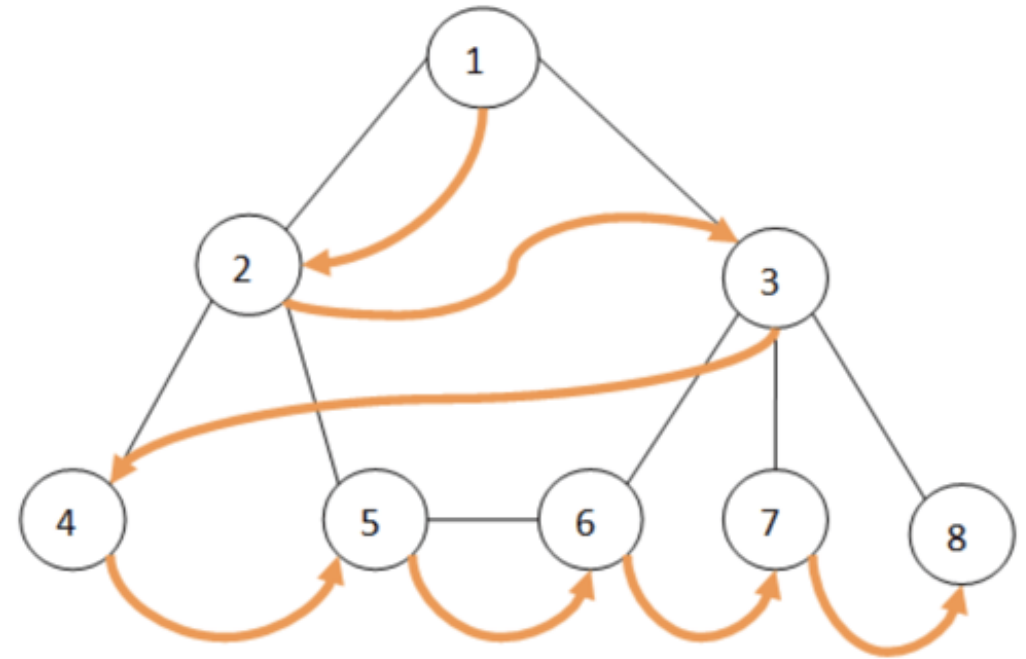
- Depth-limited Search (**Not Practical**)
- Iterative deepening depth-first search
(**Not Practical**)
- Bidirectional Search (**Not Practical**)
- Uniform Cost Search

DFS



Stack

BFS



Queue

- Definition

- A depth-limited search algorithm is similar to depth-first search with a predetermined limit ℓ . Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.
- Depth-limited search can be terminated with **two Conditions of failure**:
 - Standard failure value: It indicates that problem does not have any solution.
 - Cutoff failure value: It defines no solution for the problem within a given depth limit.

- Advantages

- Depth-limited search is Memory efficient.

- Disadvantages

- Unfortunately, if we make a poor choice for ℓ the algorithm will fail to reach the solution, making it incomplete again.
- It may not be optimal if the problem has more than one solution.

Depth-limited
Search

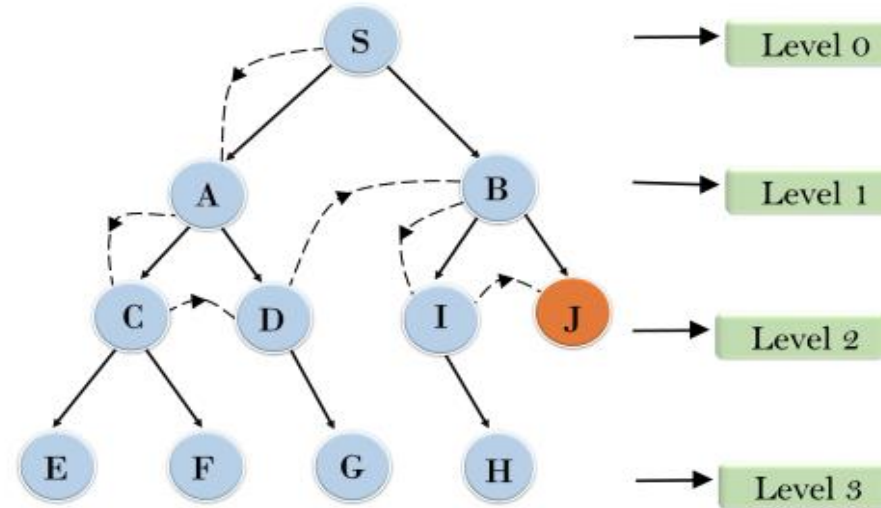
Iterative
deepening depth-
first search

Bidirectional
Search

Depth Limited Search

$\ell = 2$

S---> A--->C--->D--->B--->I--->J



function DEPTH-LIMITED-SEARCH(*problem*, ℓ) **returns** a node or *failure* or *cutoff*

frontier $\leftarrow$ a LIFO queue (stack) with NODE(*problem*.INITIAL) as an element

result $\leftarrow$ *failure*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

if DEPTH(*node*) > ℓ **then**

result $\leftarrow$ *cutoff*

else if not IS-CYCLE(*node*) **do**

for each *child* **in** EXPAND(*problem*, *node*) **do**

add *child* to *frontier*

return *result*

**Depth-limited
Search**

Iterative
deepening depth-
first search

Bidirectional
Search

Depth-limited Search

Iterative
deepening depth-
first search

Bidirectional
Search

- **Time Complexity:** Time complexity of DLS algorithm is $O(b^{\ell})$.
- **Space Complexity:** Space complexity of DLS algorithm is $O(b \times \ell)$.
- **Completeness:** DLS search algorithm is complete if the solution is above the depth-limit.
- **Optimality:** Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

- Definition

- The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.
- This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

- Advantages

- It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

- Disadvantages

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Depth-limited
Search

**Iterative
deepening
depth-first search**

Bidirectional
Search

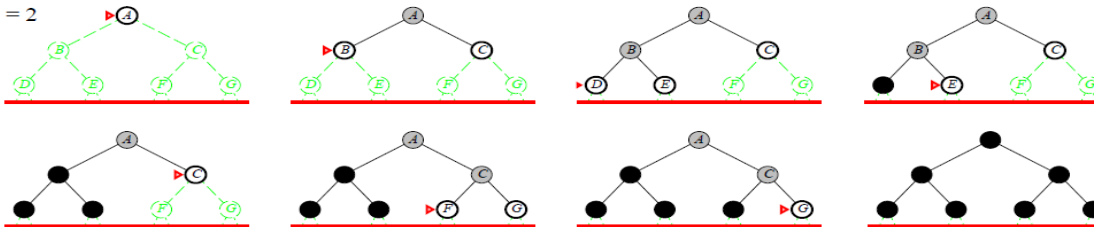
Iterative deepening search $l = 1$

Limit = 1



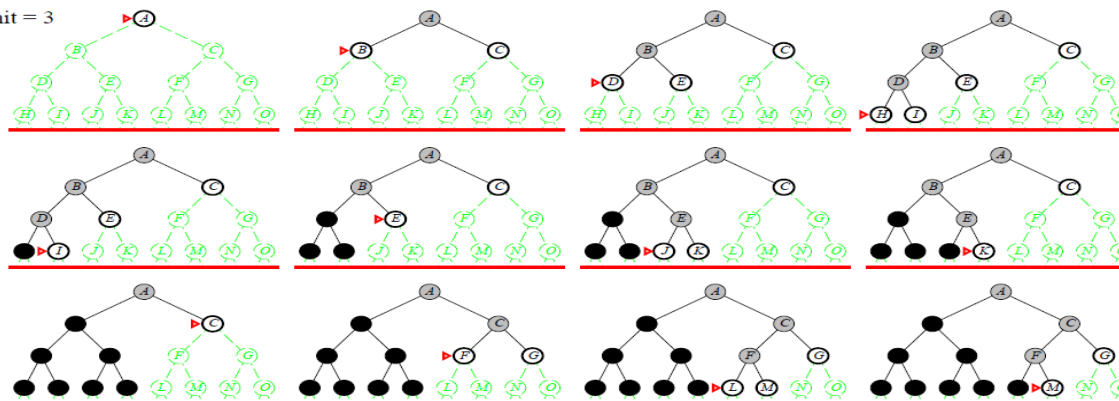
Iterative deepening search $l = 2$

Limit = 2



Iterative deepening search $l = 3$

Limit = 3

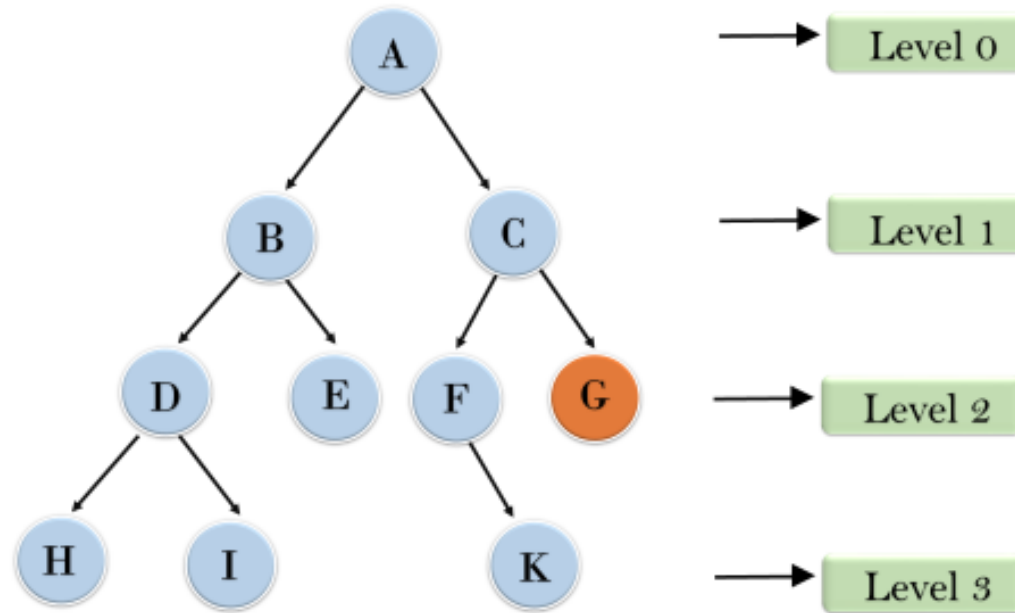


Depth-limited
Search

**Iterative
deepening
depth-first search**

Bidirectional
Search

Iterative deepening depth first search



1'st Iteration-----> A

2'nd Iteration----> A, B, C

3'rd Iteration----->A, B, D, E, C, F, G

4'th Iteration----->A, B, D, H, I, E, C, F, K, G

Depth-limited
Search

**Iterative
deepening
depth-first search**

Bidirectional
Search

Depth-limited
Search

**Iterative
deepening
depth-first search**

Bidirectional
Search

- **Time Complexity:** Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.
- **Space Complexity:** The space complexity of IDDFS will be $O(bd)$.
- **Completeness:** This algorithm is complete if the branching factor is finite

- Definition

- Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search, to find the goal node. The search stops when these two graphs intersect each other.
- Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex.

- Advantages

- Bidirectional search is fast.
- Bidirectional search requires less memory

- Disadvantages

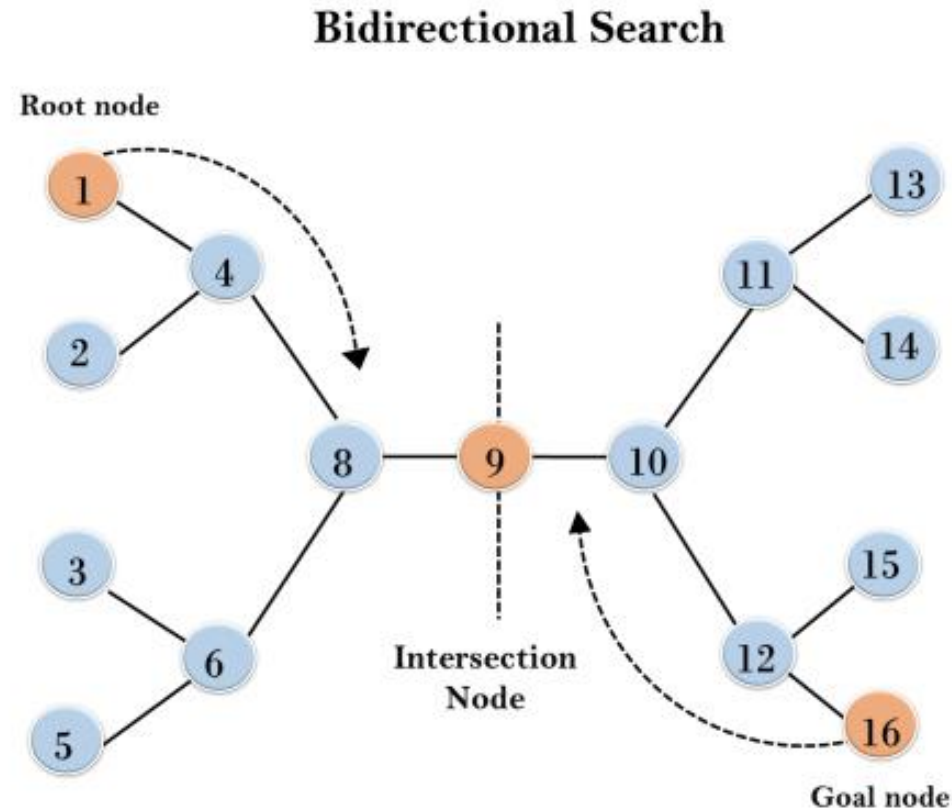
- Implementation of the bidirectional search tree is difficult.
- In bidirectional search, one should know the goal state in advance.

Depth-limited
Search

Iterative
deepening depth-
first search

**Bidirectional
Search**

- In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.
- The algorithm terminates at node 9 where two searches meet.
- Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.



Depth-limited
Search

Iterative
deepening depth-
first search

**Bidirectional
Search**

Depth-limited
Search

Iterative
deepening depth-
first search

**Bidirectional
Search**

Uniform Cost
Search

- **Time Complexity:** Time complexity of bidirectional search using BFS is $O(b^d)$.
- **Space Complexity:** Space complexity of bidirectional search is $O(b^d)$.
- **Completeness:** Bidirectional Search is complete if we use BFS in both searches.

Criterion	Breadth-First	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Search Algorithm

```
graph TD; SA([Search Algorithm]) --> UB[Uniformed/Blind]; SA --> IS[Informed Search]; UB --> B1[ ]; B1 --> B2[ ]; B2 --> UCS[Uniform cost search]; B2 --> B3[ ]; B3 --> B4[ ]; B4 --> B5[ ]; B5 --> B6[ ]; IS --> BFS[Best First Search]; IS --> AS[A*search]; style B1 fill:none,stroke:none; style B2 fill:none,stroke:none; style B3 fill:none,stroke:none; style B4 fill:none,stroke:none; style B5 fill:none,stroke:none; style B6 fill:none,stroke:none;
```

Uniformed/Blind

Uniform cost search

Informed Search

Best First Search

A*search

Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**

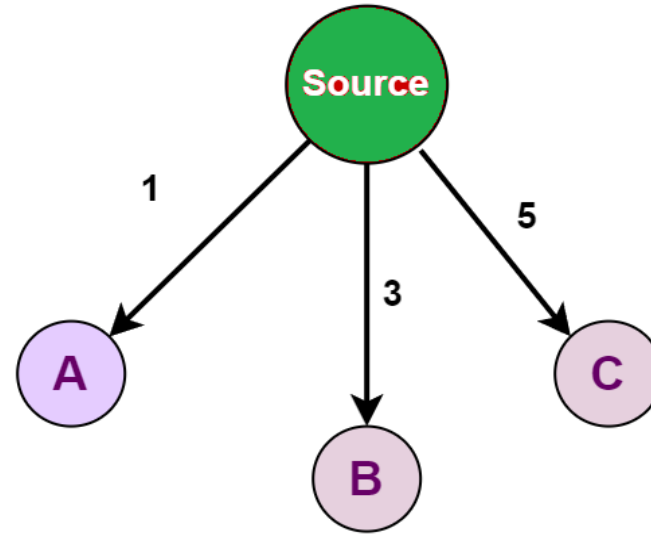
- Definition

- When all step costs are equal, breadth-first search is optimal because it always expands the shallowest unexpanded node. By a simple extension, we can find an algorithm that is optimal with any step-cost function.
- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge.
- The primary goal of the uniform-cost search is to find **a path to the goal node which has the lowest cumulative cost**. It can be used to solve any graph/tree where **the optimal cost is in demand**.
- Uniform-cost search expands nodes according to their **path costs from the root node**.

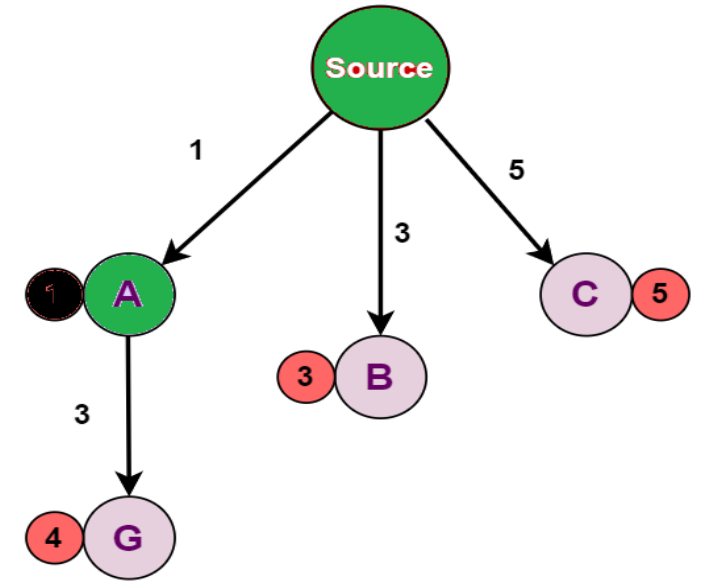
Visited Nodes: –



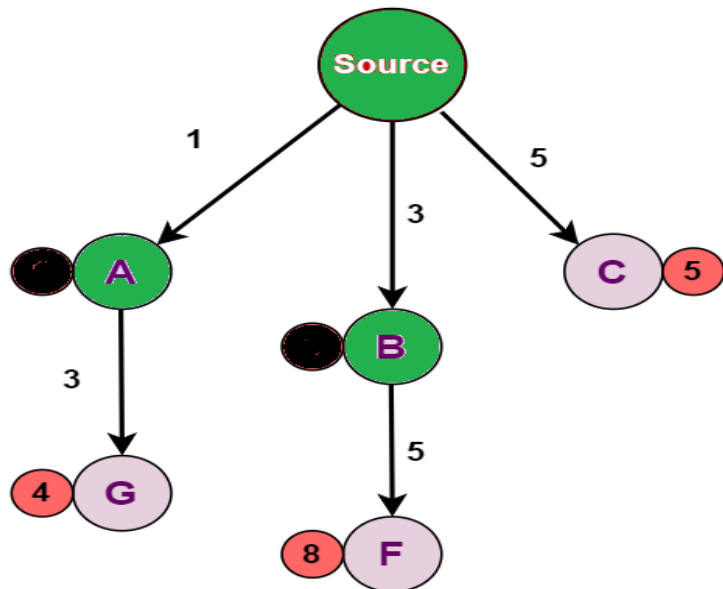
Visited Nodes: source



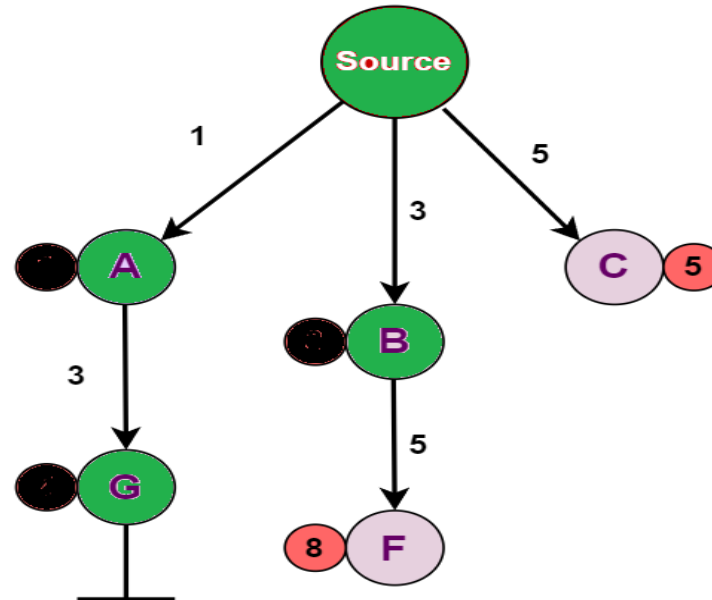
Visited Nodes: source, A



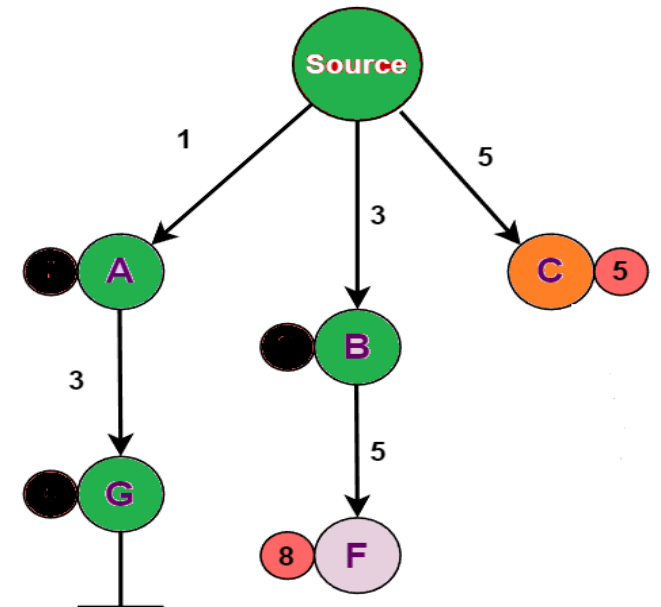
Visited Nodes: source, A, B



Visited Nodes: source, A, B, G



Visited Nodes: source, A, B, G, C



Frontier = [(S,0)]

Frontier = [(A,1) , (B,4)]

Frontier = [(D,3) , (B,4) , (C,4)]

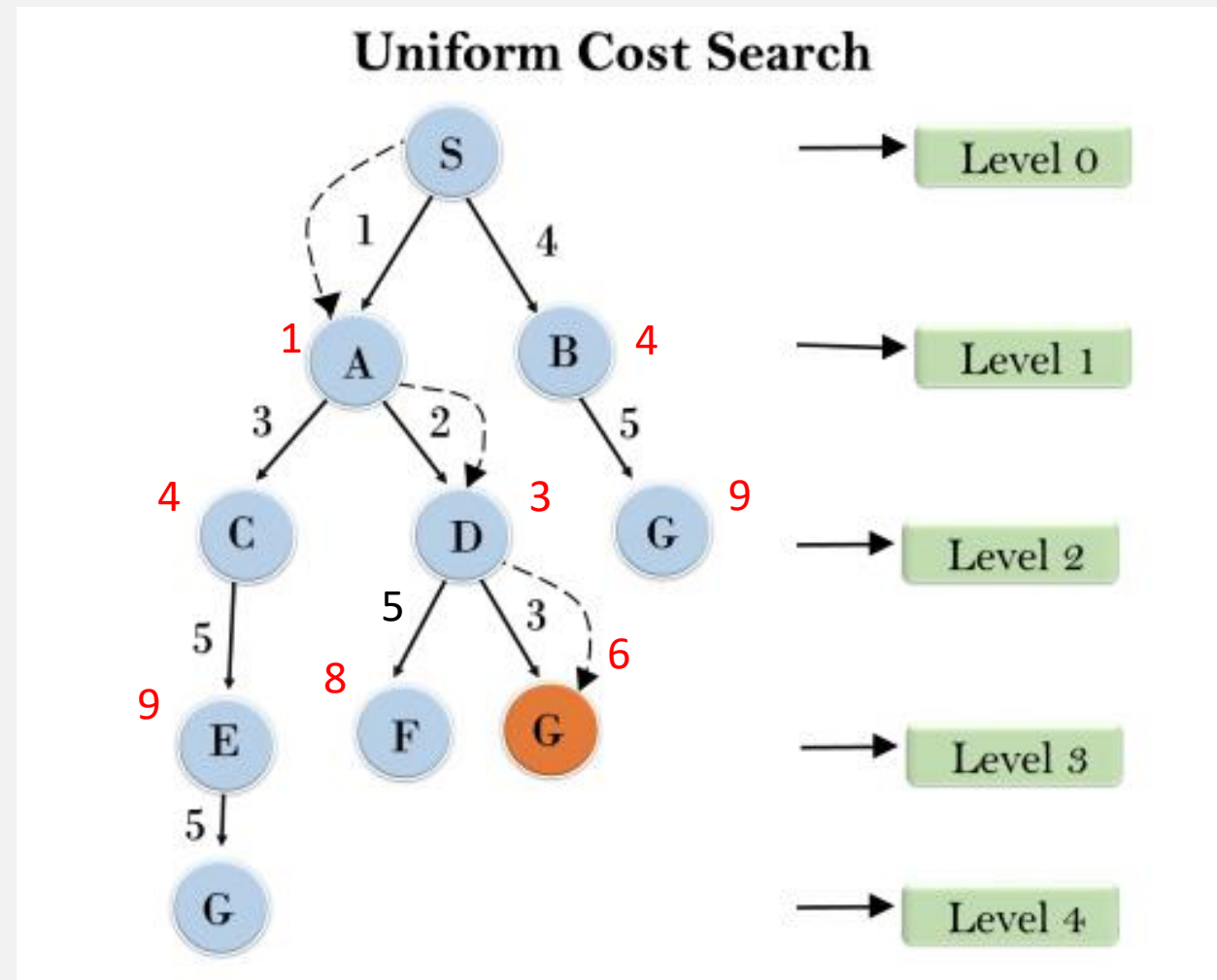
Frontier = [(B,4) , (C,4) , (G,6) , (F,8)]

(G,9) ignored

Frontier = [(C,4) , (G,6) , (F,8)]

Frontier = [(G,6) , (F,8) , (E,9)]

(G,6) Goal



S---> A--->D---->G

- A uniform-cost search algorithm is implemented by the **priority queue**.

Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**

Uniform-cost Search Pseudocode

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

frontier $\leftarrow$ INSERT(*child*, *frontier*)

else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**

Romania Problem

Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**

- Definition

- Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.
- Uniform-cost search does not care about the number of steps a path has, but only about their total cost.
- Therefore, it will get stuck in an infinite loop if there is a path with an infinite sequence of zero-cost for example, a sequence of **NoOp** actions

- Advantages

- Uniform cost search is optimal because at every state the path with the least cost is chosen.

- Disadvantages

- It does not care about the number of steps involve in searching and only concerned about path cost.

Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**

- **Time Complexity:** the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$. Let C^* is **Cost of the optimal solution**, and ϵ is each step to get closer to the goal node.
- **Space Complexity:** The same logic is for space complexity so; the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.
- **Completeness:** Uniform-cost search is complete, such as if there is a solution, UCS will find it.
- **Optimality:** Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

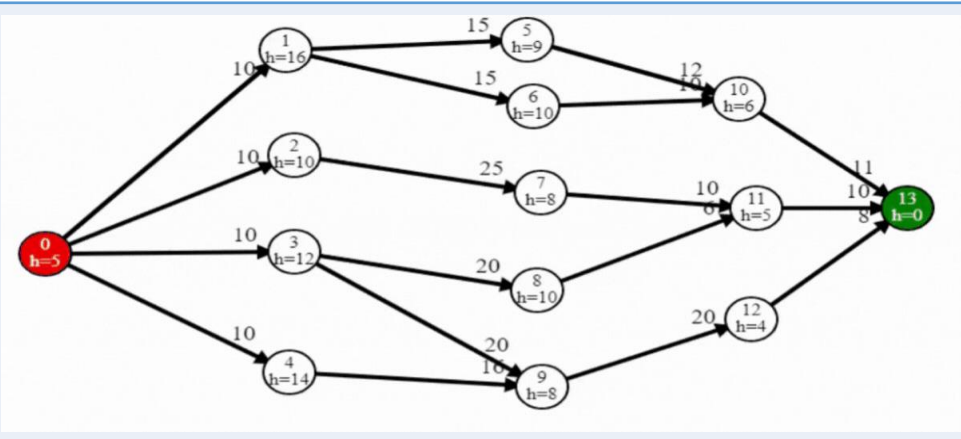
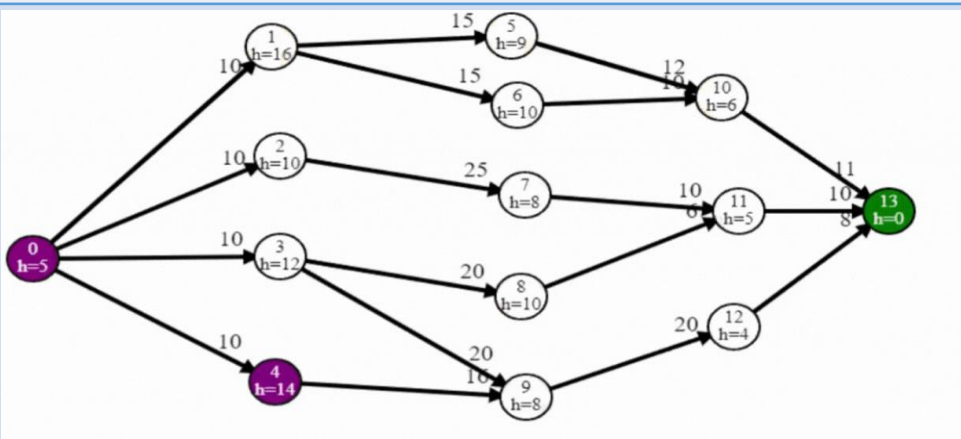
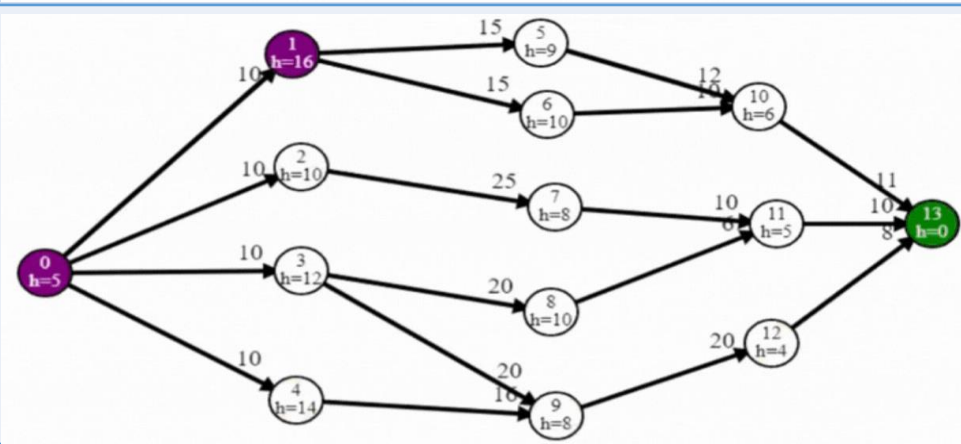
Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**

Uninformed/ Blind Search

BFS				Depth-limited Search
DFS				Iterative deepening depth-first search
UCS				Uniform Cost Search

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Depth-limited
Search

Iterative
deepening depth-
first search

Bidirectional
Search

**Uniform Cost
Search**